CareerFlux — Final Project Report and Interview Guide

Part 1 — Project report

1. What it is (one paragraph)

CareerFlux is a **college placement and career-intelligence platform**. A college (placement office, department coordinators, students) uses it to find jobs from employers that are *legitimately* readable, match students to them with a **deterministic, explainable score**, and run a placement drive end to end (shortlist → invite → interested → selected). Its differentiator is **source intelligence**: every place jobs are collected from is a governed record with a lifecycle, a policy check (robots.txt, terms, no login/CAPTCHA/paywall) and a health history, so the system can answer "why are we reading this site?".

It is not a job board, not a recruiter/ATS portal, and not a chatbot.

2. Actors (exactly four)

Actor	Owns
PORTAL_ADMIN	Platform: institutions, AI controls, source governance. No automatic access to student records
PLACEMENT_COORDINATOR	One college: departments, batches, enrolment, drives
DEPARTMENT_COORDINATOR	Department/batch-scoped students and activity; fails closed with no grant
STUDENT	Own profile, resume, skills, preferences, applications

No recruiter role: companies never sign in. They appear only as the subject of a placement requirement or as the owner of a public job board.

3. Stack

Layer	Choice
Backend	Java 21, Spring Boot 3.5, Spring Security (stateless JWT), Spring Data JPA, Flyway, Spring Mail
Database	PostgreSQL (prod) / H2 in PG-compat mode (dev). 29 Flyway migrations, portable SQL
Messaging	`PipelineEventBus` port: in-process transactional outbox (default) or Kafka (`kafka` profile)
AI	Spring AI + Gemini, behind a single `AiClient` boundary; fully optional
Frontend	React 18, TypeScript, Vite, TanStack Query, React Router, hand-written CSS design tokens
Deploy	Docker Compose (Postgres + backend + nginx frontend) or Render; `RUNBOOK.md`, `DEPLOYMENT.md`, backup script

Size: ~344 main Java files, 21 REST controllers, ~1,600 `@Test` methods, 87 frontend files.

4. Architecture in brief

```
React SPA ──/api──► Spring Boot (one deployable) ──► PostgreSQL
                      ├─ Source adapters ──► public ATS APIs
                      └─ PipelineEventBus ─► outbox (default) | Kafka
```

Backend packages: `source` (registry, policy engine, adapters, health), `ingestion` (pipeline, events), `job` (canonical jobs + observations + changes), `matching`, `candidate`, `institution`, `requirement`, `shortlist`, `consent`, `privacy/erasure`, `ai`, `audit`, `auth`, `security`, `notification`, `discovery`.

5. Key features and how they work

1. **Source state machine** — `DISCOVERED → CLASSIFIED → POLICY_REVIEW → APPROVED → ACTIVE`, with `DEGRADED`, `PENDING_REVIEW`, `BLOCKED`, `RETIRED`. Transitions enumerated in `SourceState.allowedTransitions()`; only `SourceLifecycleService` may change state; every change writes an event with actor and reason.
2. **Policy gate** (`SourcePolicyEngine`) — deterministic; unknown facts count *against* a source. robots.txt must permit the exact path, terms reviewed by a named person, no access controls, rate limit configured. Re-checked in ingestion (redundant by design).
3. **Own robots.txt parser** — group selection, longest-match Allow/Disallow, `*`/`$` wildcards, Crawl-delay; errs toward refusal.
4. **Adapters** — Greenhouse, Lever, Ashby, Breezy, SmartRecruiters, Oracle HCM, configurable JSON/page adapters, local fixture. Adapters do minimal mapping; normalisation is shared downstream.
5. **Ingestion pipeline** — fetch → normalise → deduplicate → detect changes → enrich → persist/emit → close vanished postings. Each stage emits an event.
6. **Deduplication** — five signals in descending confidence (same source id, requisition id, normalised apply URL, canonical key, Jaccard description similarity > 0.72). Duplicates are never deleted: they become another `job_observation` of the canonical `job`.
7. **Matching** — Skills 40 / Experience 20 / Role 20 / Location 10 / Seniority 10. Unevaluable dimension = neutral 70 *and says so*. Every score stores its strengths/gaps and `scorer_version`. Salary is explained, never weighted.
8. **Eligibility ≠ match score** — placement eligibility is a separate deterministic rule check (verified CGPA, batch, department, requirements). Unknown never counts as eligible.
9. **Placement workflow** — `SHORTLISTED → INVITED → INTERESTED → SELECTED`, plus `NOT_PROCEEDING` and `DECLINED`. `PlacementStage.canMoveTo(actor)` makes the actor part of the rule (only the student can say INTERESTED/DECLINED). Append-only stage history in the same transaction; `@Version` optimistic locking.
10. **AI boundary** — AI proposes, a person confirms. Resume parsing yields a reviewable *proposal*; CGPA can never be written by AI; no model call inside a DB transaction; separate quota pools; consent gate (`AiProcessingPolicy.mayProcess`) before any call; PII redacted before text is sent. App is fully functional with AI off.
11. **Privacy** — minimise, scope, record, expire, disclose. Versioned append-only consent ledger, configurable retention, erasure = tombstone + anonymisation (placement history survives), student data export, audit by user/institution/role.
12. **Security** — JWT access/refresh are distinct types, user re-read from DB every request, per-user session watermark for revocation, BCrypt 12, enumeration-resistant login, hashed single-use reset tokens, 404-not-403 for foreign resources, content-based upload validation, rate limiting, prod profile refuses to start with default secrets.
13. **SSRF protection** — `SafeUrlValidator` resolves *every* address and refuses private/loopback/link-local/metadata ranges; enforced in the transport (`SafeClientHttpRequestFactory`), manual validated redirects (max 3), size-capped reads.
14. **Company source discovery** — resolve a company name → candidate domains (deterministic, never guessed by AI) → operator confirms → board probed → source registered at `DISCOVERED` and goes through the same policy review.
15. **Bulk student import** — coordinators provision student accounts from an Excel sheet; students can add a personal email (verified) for alerts.

6. Engineering practices worth citing

- Mutation-style testing: deliberately break a protection and confirm a test fails.
- Tests that cover things where a bug matters (state machine, policy, robots, dedup, scoring, authz, isolation) rather than coverage targets.
- A real bug found by driving the UI: concurrent rematch race → fixed with a per-candidate lock.
- Destructive sweeps ship in dry-run first; retention values are configuration.
- Decisions recorded with their reasoning (`docs/architecture.md`, "Things deliberately not built").

7. Honest limitations (say these yourself before you are asked)

- **No semantic/embedding matching** — rules only; embeddings are the next step behind the same explanation contract.
- **Password reset email** — token issuance works; check current state of mail wiring before claiming it sends in production. Email requires `SPRING_MAIL_HOST`.
- **Rate limiting and caching are in-process** — correct for one node; multi-instance needs a shared counter (Redis) at that point.
- **DNS rebinding gap** in SSRF protection (validate-then-connect window); closing it needs connecting to a pinned IP.
- **Salary carries no weight** and is never currency-converted.
- **Open product questions:** whether staff may see students' saved/applied public-job activity; whether placement notes are student-visible.
- **Legal:** policies (retention, consent) are engineering values, *not* legal conclusions. Do not claim DPDP compliance.
- **README is partly stale** — it still describes three adapters and omits institutions/roles; `docs/architecture.md` is the better reference. Working tree has uncommitted matching/salary-context changes.
- Pilot-grade, not battle-tested at scale: no load testing, no multi-node, no real-college production data.

Part 2 — How to handle this project in the interview

The 60-second pitch (memorise)

> *"CareerFlux is a placement platform for colleges. The interesting part isn't the job list — it's that every job source is a governed record: before we read a site, a deterministic policy gate checks robots.txt, terms and access controls, and a named person approves it. Jobs from different sources are deduplicated into one canonical job, students get an explainable weighted match score, and the college runs the shortlist-to-selection workflow with role-based, scoped access. AI is optional and only proposes — a student confirms. I built it with Spring Boot, Postgres, Flyway and React."*

Strategy

1. **Lead with problems and decisions, not features.** Interviewers probe *why*. Prepare 5 decisions you can defend: state machine for sources, observations vs. canonical jobs, outbox/Kafka abstraction, AI-proposes-human-confirms, eligibility separate from score.
2. **Pick 2 deep-dive areas** you can explain down to the class level (suggest: ingestion/dedup and authorization/isolation). Know the files: `SourcePolicyEngine`, `JobDeduplicator`, `MatchScorer`, `PlacementStage`, `SafeUrlValidator`.
3. **Own the limitations first.** Saying "no embeddings, single-node rate limiting, here's when I'd change that" shows judgment. Being caught overclaiming is the main risk.
4. **Be precise about who wrote what.** If asked whether AI tools helped, say you used them as an assistant and that you reviewed, tested and can explain every design decision. Then *actually be able to*: read the code you will be asked about the night before.
5. **Use the vocabulary correctly:** idempotency, at-least-once delivery, transactional outbox, optimistic locking, tenant isolation, fail-closed, SSRF, enumeration resistance, IDOR.
6. **Prepare a 3-minute demo path:** sign in as a placement coordinator → registered source and its lifecycle/policy verdict → job detail showing multiple observations → student match with strengths/gaps → shortlist stage move → audit/consent view. Have screenshots as backup.
7. **Don't say:** "production-ready", "scales to millions", "compliant with DPDP/GDPR", "AI decides". Say: "pilot-ready, designed so X becomes Y at scale".

Question bank with answer skeletons

Architecture and design

- *Why a monolith?* One team, one deployable; microservices would add operational cost with no isolation benefit. Package boundaries (`source`, `matching`…) keep extraction possible later.
- *Why an event-bus abstraction instead of Kafka directly?* Kafka fits a multi-stage stream, but requiring a broker to run on a laptop kills testability. Outbox table written in the same transaction gives at-least-once + idempotency; Kafka is a profile switch.
- *What is the transactional outbox and why?* Writing the event in the same DB transaction as the data avoids the dual-write problem (data committed but event lost or vice versa).
- *Why* `jobs` *and* `job_observations` *separately?* Provenance and multi-source dedup; duplicates add observations rather than being deleted.
- *Why Flyway with portable SQL?* Reproducible schema; same migrations run on H2 and Postgres; Hibernate never generates DDL.

Data and algorithms

- *How does dedup work? What if two different jobs are merged?* Five signals by confidence; high 0.72 Jaccard threshold because wrongly merging is worse than showing a near-duplicate. Which source may overwrite fields (owner only) avoids title flapping.
- *Explain the match score.* Five weighted sub-scores; neutral 70 when not evaluable and flagged; components stored; scorer_version for traceability.
- *Why not use an LLM or embeddings for matching?* Explainability and reproducibility; AI only phrases explanations from already-computed results. Embeddings are a planned layer behind the same contract.

- *Concurrency bug you hit?* Profile-changed listener plus explicit UI rematch both did find-then-insert → unique-constraint collision; fixed with a per-candidate lock. Follow-up: a lock is single-node; at multiple nodes you'd use DB-level upsert or advisory lock.

Security and privacy (high probability, since this is the project's strength)

- *How do you prevent one college seeing another's students?* Institution and department scoping enforced in the service layer, not just controllers; coordinator with no grant sees nobody (fail closed); foreign resource → 404.
- *Why 404 instead of 403?* 403 confirms the id exists.
- *How are JWTs revoked?* Per-user watermark timestamp; tokens issued before it are rejected; the user is re-read each request so disabling takes effect immediately. Trade-off: one DB read per request.
- *Password reset design?* 256-bit SecureRandom, hashed (SHA-256) at rest, single use, 30 min, reset bumps the watermark.
- *What is SSRF and how did you defend?* Operator-typed URLs become server-side requests; validate every resolved IP, enforce at transport, no auto-redirects, size caps; state the DNS-rebinding residual gap.
- *Erasure vs. placement records?* Tombstone + anonymisation so placement history and audit integrity survive while identity and personal data are removed.
- *Is it GDPR/DPDP compliant?* "I treated those as engineering policies, not legal conclusions; compliance needs legal review."

AI

- *Where is AI used and where is it forbidden?* Used: resume reading (as a proposal), requirement extraction, explanation phrasing. Never: auth, authorization, eligibility, CGPA, source approval, scoring.
- *What if the AI is down or hallucinates?* Deterministic fallback everywhere; output is a reviewable proposal; absence is never treated as a correction; tests use a fake client.
- *Cost and abuse control?* Separate quota pools (interactive vs. ingestion), 429 stops ingestion AI for the run, consent gate and redaction before any call.

Ethics and legality of scraping

- *Isn't scraping risky?* It isn't scraping in the bypass sense: only documented public endpoints (ATS syndication), robots.txt honoured, ambiguous terms = refusal, login/CAPTCHA/paywall are permanent disqualifiers, human approval required.

Testing and quality

- *How did you test?* Focus on risk areas; mutation-style checks (break the guard, expect red); fixtures; no real AI key needed.
- *What's your weakest area?* Pick honestly: e.g. no load tests, single-node assumptions, frontend test coverage is thin (only typecheck/vitest).

Operations

- *How is it deployed?* Compose with Postgres + backend + nginx, or Render; prod profile refuses to boot on default secrets/CORS `*`; runbook and backup script; 14-day backup retention, deletion ledger reapplied after restore.
- *What would you do first to scale?* Shared rate limiter (Redis), Kafka profile, read replicas/search index if listing queries slow, observability.

Behavioural

- *Hardest problem?* Pick one with a story: policy gate design, dedup correctness, or role-model migration (renaming roles safely in order).
- *What would you do differently?* Earlier role model; end-to-end tests earlier; fewer docs drift (README stale).
- *What's next?* Embeddings, real email delivery, OAuth, Redis when multi-node, load testing.

Curveball questions and safe answers

Question	Safe answer
"Did you write all of this yourself?"	Be truthful about tool use; emphasise ownership: you made the decisions, reviewed and can walk through any path.

Question	Safe answer
"How many users?"	"Pilot-scale; no production load yet" — don't invent numbers.
"Why not just use LinkedIn/Naukri?"	They don't give a college-scoped workflow or source provenance; ToS forbids reading them, which is exactly the point of the policy gate.
"Show me the code for X"	Know entry points: `SourceLifecycleService.transition`, `JobDeduplicator`, `MatchScorer`, `PlacementStage.canMoveTo`, `SafeUrlValidator`, `AiProcessingPolicy`.
"Why Java/Spring and not Node/Python?"	Strong typing for a rule-heavy domain, mature security/JPA/Flyway ecosystem, Spring AI integration in-process.

Last-day checklist

- Start the app (`dev,demo` profile) and rehearse the demo twice.
- Re-read `docs/architecture.md` and this file.
- Open and skim the five key classes listed above.
- Rehearse the pitch, the limitations list, and two deep dives out loud.
- Confirm what is committed vs. uncommitted; do not demo from a broken tree.
- Prepare 2 questions to ask the interviewer (team's approach to tenant isolation, how they handle third-party data sources).